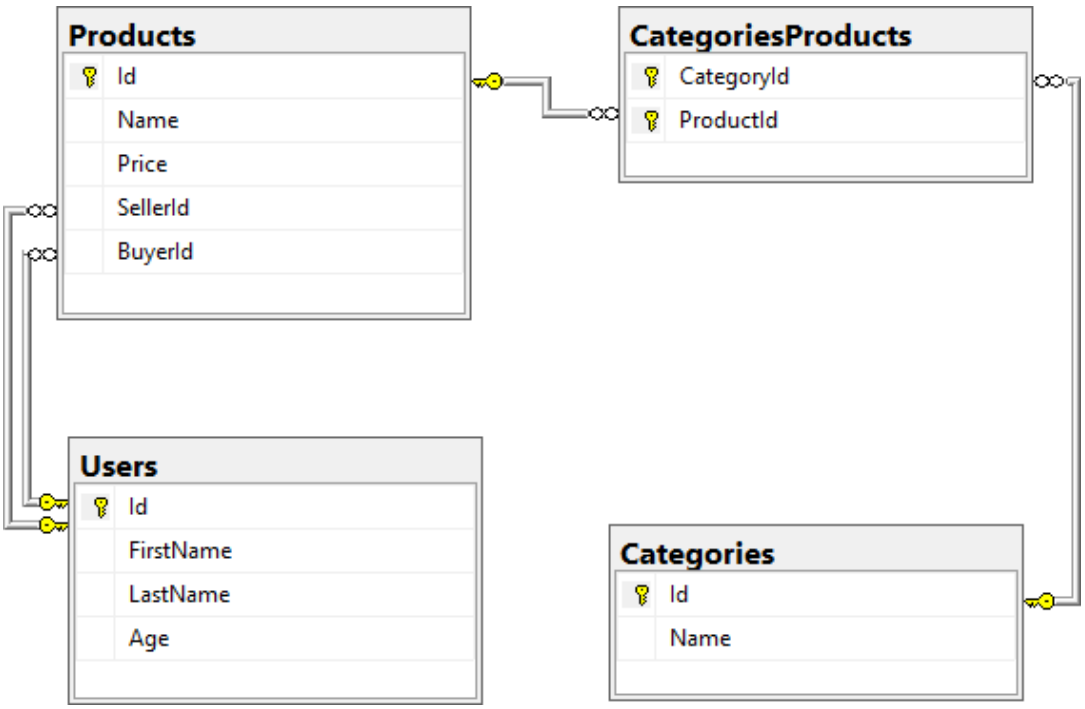
Exercises: External Format Processing

You can check your solutions in Judge

Products Shop Database

A products shop holds **users**, **products** and **categories** for the products. Users can **sell** and **buy** products.

- **`Users`** have an **`Id`**, **`FirstName`** (optional), **`LastName`** and **`Age`** (optional).
- **`Products`** have an **`Id`**, **`Name`**, **`Price`**, **`BuyerId`** (optional) and **`SellerId`** as **`Id`**s of **`Users`**.
- **`Categories`** have an **`Id`** and **`Name`**.
- Using Entity Framework and Code First create a database, following the above description.



- **`Users`** should have **many `Products` sold** and **many `Products` bought**.
- **`Products`** should have **many `Categories`**.
- **`Categories`** should have **many `Products`**.
- **`CategoriesProducts`** should **map `Products`** and **`Categories`**.

1. Import Data

Query 1. Import Users

NOTE: You will need method **`public static string ImportUsers(ProductShopContext context, string inputJson)`** and **`public StartUp`** class.

Import the users from the provided file "**`users.json`**".

Your method should return a string with the following message:
`$"Successfully imported {users.Count}";`

Query 2. Import Products

NOTE: You will need method **`public static string ImportProducts(ProductShopContext context, string inputJson)`** and **`public StartUp`** class.

Import the users from the provided file "**`products.json`**".

Your method should return a string with the following message:
`$"Successfully imported {products.Count}";`

Query 3. Import Categories

NOTE: You will need method **`public static string ImportCategories(ProductShopContext context, string inputJson)`** and **`public StartUp`** class.

Import the users from the provided file "**`categories.json`**". Some of the names will be null, so you don't have to add them to the database. Just skip the record and continue.

Your method should return a string with the following message:
`$"Successfully imported {categories.Count}";`

Query 4. Import Categories and Products

NOTE: You will need method **`public static string ImportCategoryProducts(ProductShopContext context, string inputJson)`** and **`public StartUp`** class.

Import the users from the provided file "**`categories-products.json`**".

Your method should return a string with the message:
`$"Successfully imported {categoryProducts.Count}";`

2. Export Data

Write the below-described queries and **export** the returned data to the specified **format**. Make sure that Entity Framework Core generates only a **single query** for each task.

Note that because of the random generation of the data, the output probably will be different.

Query 5. Export Products in Range

NOTE: You will need method **`public static string GetProductsInRange(ProductShopContext context)`** and **`public StartUp`** class.

Get all products in a specified **price range:** 500 to 1000 (inclusive). Order them by price (from lowest to highest). Select only the **product name**, **price** and the **full name of the seller**. Export the result to JSON.

products-in-range.json

```
[
  {
    "name": "TRAMADOL HYDROCHLORIDE",
    "price": 516.48,
    "seller": " Thompson"
  },
  {
    "name": "Allopurinol",
    "price": 518.50,
    "seller": "Jacqueline Perez"
  },
```

```
  {
    "name": "Parsley",
    "price": 519.06,
    "seller": "Brenda Howell"
  },
...
]
```

Query 6. Export Sold Products

NOTE: You will need method **`public static string GetSoldProducts(ProductShopContext context)`** and **`public StartUp`** class.

Get all users who have **at least 1 sold item** with a **buyer**. Order them by **the last name**, then by **the first name**. Select the person's **first** and **last name**. For each of the **sold products** (products with buyers), select the product's **name**, **price** and the buyer's **first** and **last name**.

users-sold-products.json

```
[
  {
    "firstName": "Gloria",
    "lastName": "Alexander",
    "soldProducts": [
      {
        "name": "Metoprolol Tartrate",
        "price": 1405.74,
        "buyerFirstName": "Bonnie",
        "buyerLastName": "Fox"
      }
    ]
  },
...
]
```

Query 7. Export Categories by Products Count

NOTE: You will need method **`public static string GetCategoriesByProductsCount(ProductShopContext context)`** and **`public StartUp`** class.

Get **all categories**. Order them in descending order by the category's **products count**. For each category select its **name**, the **number of products**, the **average price of those products** (rounded to the second digit after the decimal separator) and the **total revenue** (total price sum and rounded to the second digit after the decimal separator) of those products (regardless if they have a buyer or not).

categories-by-products.json

```
[
  {
    "category": "Garden",
```

```
    "productsCount": 23,
    "averagePrice": "800.15",
    "totalRevenue": "18403.47",
  },
  {
    "category": "Weapons",
    "productsCount": 22,
    "averagePrice": "671.07",
    "totalRevenue": "14763.61"
  },
...
]
```


Query 8. Export Users and Products

NOTE: You will need method **`public static string GetUsersWithProducts(ProductShopContext context)`** and **`public StartUp`** class.

Get all users who have **at least 1 sold product** with a **buyer**. Order them in descending order by the **number of sold products to a buyer**. Select only their **first** and **last name** and **age** and for each product – **name** and **price**. Ignore all null values.

Export the results to **JSON**. Follow the format below to better understand how to structure your data.

users-and-products.json


```
{
"usersCount": 54,
"users": [
  {
    "lastName": "Stewart",
    "age": 39,
    "soldProducts":
    {
      "count": 9,
      "products":
      [
      {
        "name": "Finasteride",
        "price": 1374.01
      },
      {
        "name": "Glyburide",
        "price": 95.10
```

```
      },
      {
        "name": "GOONG SECRET CALMING BATH ",
        "price": 742.47
      },
      {
        "name": "EMEND",
        "price": 1365.51
      },
      {
        "name": "Allergena",
        "price": 109.32
      },
      ...
      ]
    }
  },
 ...
]
}
```

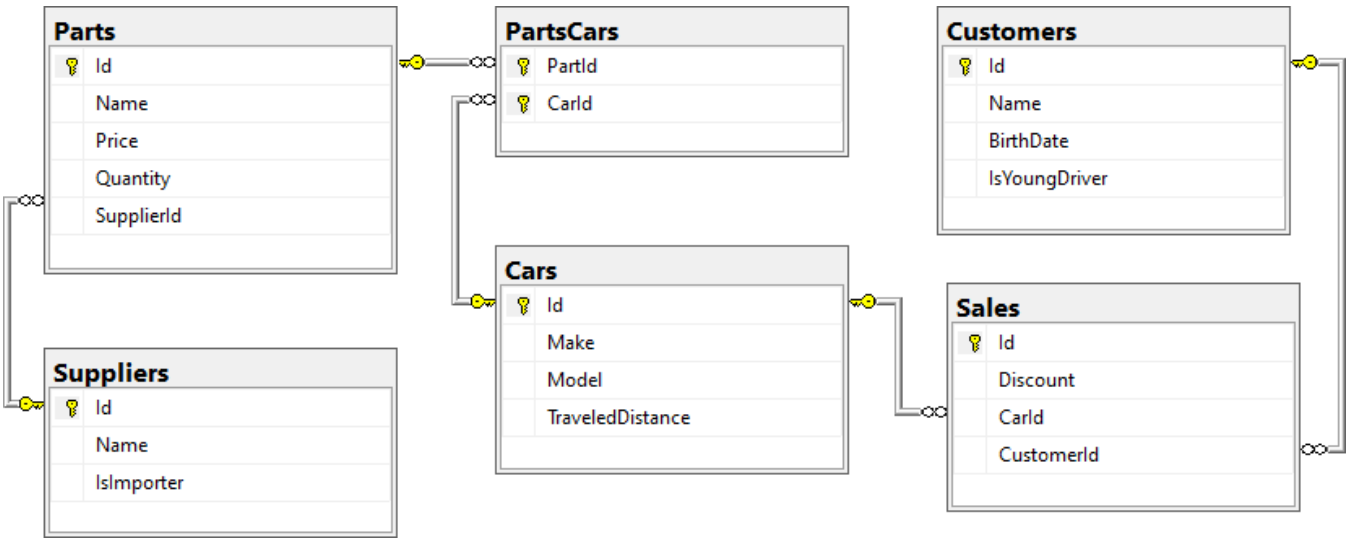

Car Dealer

1. Setup Database

A car dealer needs information about cars, their parts, parts suppliers, customers and sales.

- **Cars** have **Make, Model**, **TraveledDistance** in kilometers.
- **Parts** have **Name**, **Price** and **Quantity.**
- **Supplier** has a **Name** and info whether they supply **imported parts.**
- **Customer** has a **Name**, **BirthDate** and info whether they are **a young driver** (young driver is a driver that has **less than 2 years of experience**. Those customers get **an additional 5% off** for the sale.).
- **Sale** has **a Car**, **Customer** and a **Discount percentage.**

A **Price of a Car** is formed by **the total price** of **its Parts**.



- A **Car** has **many Parts** and **one Part** can be placed **in many Cars.**
- **One Supplier** can supply **many Parts** and each **Part** can be delivered by **only one Supplier.**
- In **one Sale**, only **one Car** can be sold to only one **Customer**.
- **A Customer** can buy **many Cars.**

2. Import Data

Import data from the provided files ("**suppliers.json**", "**parts.json**", "**cars.json**", "**customers.json**")

Query 9. Import Suppliers

NOTE: You will need method **public static string ImportSuppliers(CarDealerContext context, string inputJson)** and **public StartUp** class.

Import the suppliers from the provided file "**suppliers.json**".

Your method should return a string with the following message:
$"Successfully imported {suppliers.Count}.";

Query 10. Import Parts

NOTE: You will need method **public static string ImportParts(CarDealerContext context, string inputJson)** and **public StartUp** class.

Import the parts from the provided file "**parts.json**". If the **supplierId** doesn't exist in the **Suppliers** table, skip the record.

Your method should return a string with the following message:
$"Successfully imported {parts.Count}.";

Query 11. Import Cars

NOTE: You will need method **public static string ImportCars(CarDealerContext context, string inputJson)** and **public StartUp** class.

Import the cars from the provided file "**cars.json**".

Your method should return string with the following message:
$"Successfully imported {cars.Count}.";

Query 12. Import Customers

NOTE: You will need method **`public static string ImportCustomers(CarDealerContext context, string inputJson)`** and **`public StartUp`** class.

Import the customers from the provided file "**`customers.json`**".

Your method should return a string with the following message:
`$"Successfully imported {customers.Count}.";`

Query 13. Import Sales

NOTE: You will need method **`public static string ImportSales(CarDealerContext context, string inputJson)`** and **`public StartUp`** class.

Import the sales from the provided file "**`sales.json`**".

Your method should return a string with the following message:
`$"Successfully imported {sales.Count}.";`

3. Export Data

Write the below described queries and **export** the returned data to the specified **format**. Make sure that Entity Framework Core generates only a **single query** for each task.

Query 14. Export Ordered Customers

NOTE: You will need method **`public static string GetOrderedCustomers(CarDealerContext context)`** and **`public StartUp`** class.

Get all **customers** ordered by their **birth date ascending**. If two customers are born on the same date **first print those who are not young drivers** (e.g., print experienced drivers first). **Export** the list of customers **to JSON** in the format provided below.

ordered-customers.json

```
[
  {
    "Name": "Louann Holzworth",
    "BirthDate": "01/10/1960",
    "IsYoungDriver": false
  },
  {
    "Name": "Donnetta Soliz",
    "BirthDate": "01/10/1963",
    "IsYoungDriver": true
  },
...
]
```

Query 15. Export Cars from Make Toyota

NOTE: You will need method **`public static string GetCarsFromMakeToyota(CarDealerContext context)`** and **`public StartUp`** class.

Get all **cars** with **`Toyota`** make and **order them by model alphabetically** and by **traveled distance descending**. **Export** the list of **cars to JSON** in the format provided below.

toyota-cars.json

```
[
  {
    "Id": 134,
    "Make": "Toyota",
    "Model": "Camry Hybrid",
    "TraveledDistance": 486872832,
  },
  {
    "Id": 139,
    "Make": "Toyota",
    "Model": "Camry Hybrid",
    "TraveledDistance": 397831570,
  },
...
]
```

Query 16. Export Local Suppliers

NOTE: You will need method **`public static string GetLocalSuppliers(CarDealerContext context)`** and **`public StartUp`** class.

Get all suppliers that do not import parts from abroad. Get their id, name and the number of parts they can offer to supply. Export the list of suppliers to JSON in the format provided below.

local-suppliers.json

```
[
  {
    "Id": 2,
    "Name": "Agway Inc.",
    "PartsCount": 3
  },
  {
    "Id": 4,
```

```
    "Name": "Airgas, Inc.",
    "PartsCount": 2
  },
...
]
```

Query 17. Export Cars with Their List of Parts

NOTE: You will need method **`public static string GetCarsWithTheirListOfParts(CarDealerContext context)`** and **`public StartUp`** class.

Get all **cars along with their list of parts**. For the **car** get only **make, model** and **traveled distance** and for the **parts** get only **name** and **price** (formatted to 2nd digit after the decimal point). **Export** the list of **cars and their parts to JSON** in the format provided below.

cars-and-parts.json

```
[
  {
    "car": {
      "Make": "Opel",
      "Model": "Omega",
      "TraveledDistance": 176664996
    },
    "parts": [
      {
        "Name": "Rear Right Side Inner door handle",
        "Price": "79.99"
      },
      {
        "Name": "Door water-shield",
        "Price": "123.99"
      },
      {
        "Name": "Front Right Side Door Glass",
        "Price": "100.91"
      },
      {
        "Name": "Window motor",
        "Price": "123.49"
      },
      {
        "Name": "Rocker arm",
        "Price": "98.99"
      },
      {
        "Name": "Turbocharger",
        "Price": "0.40"
      },
      {
        "Name": "Water pipe",
        "Price": "44.94"
      },
      {
        "Name": "Muffler",
```

```
        "Price": "106.90"
      },
      {
        "Name": "Heat shield",
        "Price": "10.99"
      },
      {
        "Name": "Speed reducer",
        "Price": "14.99"
      },
      {
        "Name": "Differential seal",
        "Price": "109.99"
      }
    ]
  },
  {
    "car": {
      "Make": "Opel",
      "Model": "Astra",
      "TraveledDistance": 516628215
    },
    "parts": [
      {
        "Name": "Front Left Side Door Glass",
        "Price": "100.92"
      },
      {
        "Name": "Fan belt",
        "Price": "10.99"
      },
      {
        "Name": "Tappet",
        "Price": "300.29"
      }
    ]
  },
  {
    "car": {
      "Make": "Opel",
      "Model": "Astra",
      "TraveledDistance": 156191509
    },
    "parts": [
      {
        "Name": "Sunroof Rail",
        "Price": "100.25"
      },
      {
        "Name": "Window seal",
        "Price": "100.99"
      },
      {
        "Name": "Steering box",
        "Price": "103.99"
      }
    ]
```

```
  },
...
]
```

Query 18. Export Total Sales by Customer

NOTE: You will need method **`public static string GetTotalSalesByCustomer(CarDealerContext context)`** and **`public StartUp`** class.

Get all customers that have bought at least 1 car and get their names, bought cars count and total spent money on cars. Order the result list by total spent money descending, then by total bought cars again in descending order. **Export** the list of **customers** to **JSON** in the format provided below.

customers-total-sales.json

```
[
  {
    "fullName": " Faustina Burgher",
    "boughtCars": 4,
    "spentMoney": 12585.89
  },
  {
    "fullName": " Garret Capron",
    "boughtCars": 3,
    "spentMoney": 11743.59
 },
  {
    "fullName": " Carri Knapik",
    "boughtCars": 4,
    "spentMoney": 11550.63
  },
...
]
```

Query 19. Export Sales with Applied Discount

NOTE: You will need method **`public static string GetSalesWithAppliedDiscount(CarDealerContext context)`** and **`public StartUp`** `class`.

Get first 10 **sales** with information about the **car**, **customer** and **price** of the sale **with and without discount**. **Export** the list of sales **to JSON** in the format provided below.

sales-discounts.json

```
 [
  {
    "car": {
```

```
      "Make": "Toyota",
      "Model": "Tacoma",
      "TraveledDistance": 431663130
    },
    "customerName": "Ann Mcenaney",
    "discount": "30.00",
    "price": "195.97",
    "priceWithDiscount": "137.18"
  },
  {
    "car": {
      "Make": "Ferrari",
      "Model": "275",
      "TraveledDistance": 448008546
    },
    "customerName": "Faustina Burgher",
    "discount": "50.00",
    "price": "2547.57",
    "priceWithDiscount": "1273.79"
  },
...
]
```